WEEKLY MINI PROJECT 03 · TEAM 4

# Who Doesn't Know?

## Work Package · Backend B

Backend | NestJS + MongoDB

| PROJECT | TEAM LEADER | OWNER |
|---|---|---|
| **Who Doesn't Know?** | **Taima Watfeh** | **Ali Mikdad** |

| ROLE | AREA |
|---|---|
| **Backend developer** | **Gameplay engine** |

*The rules of the game: who asks, when a round ends, when the group decides, who wins.*

**Use the same class pattern:** keep gameplay inside `src/gameplay/`. Build `gameplay.module.ts`, `gameplay.controller.ts`, `gameplay.service.ts`, `dto/`, `schemas/` and `rules.ts`. The controller handles routes and Swagger; the service reads and writes Mongoose models; pure rules stay in testable functions.

## 1 Your Work at a Glance

| # | Task | Main file(s) |
|---|---|---|
| T1 | Pure rule functions + unit tests | `gameplay/rules.ts` |
| T2 | Dev seed script (your own test data) | `scripts/seed-dev-game.ts` |
| T3 | Ask a question | `gameplay/*` |
| T4 | Answer + turn and round progression | `gameplay/*` |
| T5 | Round decision (Continue / Vote) | `gameplay/*` |
| T6 | Suspect voting, ties, winner | `gameplay/*` |
| T7 | GameplayReadService (real) | `gameplay-read.service.ts` |
| T8 | Logger + Swagger | `gameplay/*` |

**Use the class pattern:** your gameplay module should look like the current Projects/Workspaces modules: controller for routes and Swagger, DTOs for validation, service for rules and Mongoose reads/writes, and Logger for meaningful events. Put pure game rules in small testable functions.

## 2 Your Tasks, Step by Step

### T1 Pure rule functions + unit tests

`backend/src/gameplay/rules.ts` · `rules.spec.ts`

1. Write the five functions below with no database access.
2. Test: turn wrap-around, even-round detection, strict majority, 50/50 tie, single accused, vote tie, both winner cases.

```
advanceTurn(currentTurnIndex, turnsCompletedInRound, playerCount)
  -> { currentTurnIndex, turnsCompletedInRound, roundCompleted }
shouldEnterDecision(completedRound: number): boolean          // completedRound % 2 === 0
resolveDecision(choices: ('continue'|'vote')[]): 'vote' | 'continue'   // vote > half, else continue
resolveVotes(votes: { voterRef, suspectRef }[]): { accusedRef } | { tiedRefs: number[] }
decideWinner(accusedRef, hiddenPlayerRef): 'informed' | 'hidden'
```

✔ Done when: `npm test` passes for every rule.

### T2 Dev seed script (your own test data)

`backend/scripts/seed-dev-game.ts`

1. Insert one game with room code `TEST1` in `questioning`, round 1, with 5 players.
2. Fixed temp ids `p_test1 … p_test5`, a hidden player and a tip, using the Day-0 schemas.
3. Option `--reset` to delete and recreate it.

✔ Done when: you can test all your endpoints in Swagger without Ali Al Hamwi's create / join / start.

### T3 Ask — POST /games/:roomCode/questions

`gameplay.controller.ts` · `gameplay.service.ts` · `dto/ask-question.dto.ts`

1. `AskQuestionDto { playerId, targetRef, text }`.
2. Status must be `questioning` (else 409); player must be the current player.
3. Target ≠ self and in this game; text not empty; no pending question (else 409).
4. Save the question with `round, askerRef, targetRef`; set `game.pendingQuestionId`.

✔ Done when: out-of-turn and self-questions are rejected.

### T4 Answer — POST /games/:roomCode/questions/:questionId/answer

`gameplay.service.ts` · `dto/answer-question.dto.ts`

1. Only the target answers; answer not empty.
2. Save the answer, clear `pendingQuestionId`, call `advanceTurn`.
3. If the round completed: log it; if `shouldEnterDecision` → `round_decision`, else `roundNumber + 1`.

✔ Done when: after every player asked once the round number increases; after round 2 the status is `round_decision`.

### T5 Round decision — POST /games/:roomCode/decisions

`gameplay.service.ts` · `dto/decision.dto.ts`

1. `DecisionDto { playerId, choice: 'continue' | 'vote' }` with `@IsIn`.
2. Only in `round_decision`; one choice per player per `decisionNumber` (409 on second).
3. When all chose: `resolveDecision` → `voting`, or `questioning` in the next round. Save `lastDecisionOutcome`; `decisionNumber + 1`.

✔ Done when: Flow D passes: tie continues to round 3, majority after round 4 starts voting.

### T6 Suspect voting — POST /games/:roomCode/votes

`gameplay.service.ts` · `dto/vote.dto.ts`

1. Only in `voting`; one vote per `votingNumber`; no self-vote; target in game.
2. When all voted: `resolveVotes`. Tie → save `lastVoteTie`, `votingNumber + 1`, `questioning` next round.
3. Accused → `decideWinner`, save `winner`, `accusedRef`, `status 'finished'`.
4. Every action on a finished game → 409.

✔ Done when: Flow E passes: tied vote resumes, later vote gives the right winner.

### T7 GameplayReadService (real implementation)

`backend/src/gameplay/gameplay-read.service.ts`

1. Implement the frozen interface below; replace Hamwi's stub in the module providers (same token).
2. While open, return only **counts** of submissions, never who chose / voted what.

```
// backend/src/gameplay/gameplay-read.service.ts  (frozen interface)
export interface GameplayReadService {
  getLog(gameId: number): Promise;
  getPendingQuestion(gameId: number): Promise<{ id; askerRef; targetRef; text } | null>;
  getDecisionProgress(game: GameDoc): Promise;
  getVotingProgress(game: GameDoc): Promise;
  getVoteCounts(gameId: number, votingNumber: number): Promise<{ ref: number; votes: number }[]>;
}
// Ali Al Hamwi ships a stub that returns empty values; Ali Mikdad replaces the body. Same
signature.
```

✔ Done when: the public state shows the log, counts and final vote counts.

### T8 Logger + Swagger

`gameplay/*`

1. Log: round completed, decision outcome, vote outcome / tie, final result. Never the tip or hidden player before finished.
2. Swagger tag `gameplay`, summaries and DTOs on all 4 endpoints.

✔ Done when: logs are clean and Swagger documents all endpoints.

## 3 Rules for You

- Only edit `backend/src/gameplay/*` and `backend/scripts/seed-dev-game.ts`. Follow the existing module/controller/service/dto/schema layout.
- You don't need Hamwi: rules + seed script let you build and test everything yourself.
- All endpoints return `{ ok: true }`; the frontend reads the new state by polling.

## 4 Presentation Part

Your part in the English presentation: **the business rules: turns, rounds, decisions, votes and both tie cases**. Prepare 1–2 minutes and be ready to explain your own code.